



UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN
GRÁFICA E INTERACCIÓN HUMANO
COMPUTADORA



REPORTE DE PRÁCTICA N° 9

NOMBRE: ANDREA MATA RAMÍREZ

N° CUENTA: 319052277

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 03 / 05 / 2026

CALIFICACIÓN: _____

ACTIVIDAD:

Editar Textura y blending para el humo

Animación del humo: no sale al inicio, se calienta el Aeolipile y empieza a salir, crece después de un tiempo y al apagarse el fuego el humo va decreciendo hasta ya no salir.

Brazo/palanca que lance la esfera metálica

El humo crecido inicia el movimiento del brazo palanca

Animación del brazo para lanzar

Esfera metálica

Movimiento de la esfera metálica por medio de física básica: tiro parabólico y rebote

Al dar el último rebote la esfera entra en un "hueco" en el piso y reaparece en el brazo

DESARROLLO:

Como en las prácticas anteriores se hizo la carga de modelos y texturas necesarias, en este caso se cargó una catapulta, una pelota, con sus respectivas texturas y dos texturas más, una de fuego y otra de humo.

En esta práctica el objetivo es generar una animación compleja, por lo que se usó una máquina de estados para controlar la secuencia del mecanismo que se quería implementar. La lógica es la siguiente (Figura 1)

```
/*
* Para la animación compleja se utilizó una máquina de estados
* Estados:
* 0 : Reposo
* 1 : Se enciende el fuego
* 2 : La bola gira con el fuego aun encendido
* 3 : Comienza a salir humo que crece + fuego + movimiento de bola
* 4 : Se apaga el fuego mientras el humo crece hasta un máximo
* 5 : En ese máximo se activa la catapulta y lanza la bola, humo se apaga
* 6 : Pelota lanzada, rebota y regresa a la catapulta
*/
```

Figura 1. Lógica de máquina de estados

Se implementaron varias funciones (Figura 2), entre ellas, la función principal "ActualizarEstados", esta función era la encargada de incrementar tiempos y decidir cuándo se cambia al siguiente estado, las demás funciones se implementaron para realizar las acciones que se pedían, por ejemplo:

ActualizarCatapulta: se implementó para que la catapulta se moviera solo 45° y que cambiara en el estado 4

ActualizarHumo: se implementó para generar el efecto de que el humo crecía, también para que cuando el humo llegara a su máxima longitud se activara la catapulta y una vez activada, en esta misma función se generaba el efecto de disipación del humo.

ActualizarAeolipile: Se implementó para que solo girara la bola en estado 2.

```

void ActualizarAeolipile(float& rotacionBola, int estado, float deltaTime)
{
    if (estado >= 2)
    {
        rotacionBola -= 10.0f * deltaTime;
        if (rotacionBola >= 360.0f) rotacionBola -= 360.0f;
    }
}

void ActualizarCatapulta(float& anguloBrazo, int estado, float deltaTime)
{
    if (estado == 4 && anguloBrazo < 45.0f)
    {
        anguloBrazo += 60.0f * deltaTime;
        if (anguloBrazo > 45.0f) anguloBrazo = 45.0f;
    }
}

void ActualizarHumo(float& humoEscala, int& estado, float& tiempoEstado, bool& bolaActiva, float& bola_t, float deltaTime)
{
    if (estado == 3 || estado == 4)
    {
        // humo crece
        humoEscala += 0.15f * deltaTime;
        if (humoEscala >= 4.5f)
        {
            humoEscala = 4.5f;
            // catapulta se activa
            if (estado == 4)
            {
                estado = 5;
                tiempoEstado = 0.0f;
            }
        }
    }
    else if (estado == 5)
    {
        // humo desaparece
        humoEscala -= 0.15f * deltaTime;
        if (humoEscala < 0.0f) humoEscala = 0.0f;
    }
    else if (estado == 0 || estado == 1 || estado == 2)
    {
        humoEscala = 0.0f;
    }
}

void ActualizarBola(int& estado, bool& bolaActiva, float& bola_t, glm::vec3& bolaPos, bool& bolaEnHoyo, const glm::vec3& bolaLanzada, float deltaTime)
{
    if (!bolaActiva) return;
    bola_t += deltaTime;
    const float T_vuelo = 200.0f;
    const float T_rebote = 80.0f;
    const glm::vec3 BOLA_DESTINO(bolaLanzada.x - 30.0f, -2.0f, bolaLanzada.z + 2.0f); // aterrizaje
    const glm::vec3 BOLA_REBOTE(BOLA_DESTINO.x - 5.0f, -2.0f, BOLA_DESTINO.z); // rebote

    if (!bolaEnHoyo)
    {
        // canasta hasta piso
        float t = bola_t / T_vuelo;
        if (t > 1.0f) t = 1.0f;
        bolaPos.x = bolaLanzada.x + (BOLA_DESTINO.x - bolaLanzada.x) * t;
        bolaPos.y = bolaLanzada.y + (BOLA_DESTINO.y - bolaLanzada.y) * t + 8.0f * sin(3.14159f * t); // altura del arco
        bolaPos.z = bolaLanzada.z + (BOLA_DESTINO.z - bolaLanzada.z) * t;
        if (bola_t >= T_vuelo)
        {
            bolaPos = BOLA_DESTINO;
            bolaEnHoyo = true;
            bola_t = 0.0f;
        }
    }
    else
    {
        // rebote
        float t = bola_t / T_rebote;
        if (t > 1.0f) t = 1.0f;

        bolaPos.x = BOLA_DESTINO.x + (BOLA_REBOTE.x - BOLA_DESTINO.x) * t;
        bolaPos.z = BOLA_DESTINO.z;
        bolaPos.y = -2.0f + 2.5f * sin(3.14159f * t);

        // regresa a canasta
        if (bola_t >= T_rebote)
        {
            bolaPos = bolaLanzada;
            bolaActiva = false;
            bolaEnHoyo = false;
            estado = 0;
        }
    }
}

```

Figura 2. Funciones

La parte de implementar estas funciones fue la más tardada ya que al ser una máquina de estados, se tenía un sistema en cascada y si se modificaba, por ejemplo, la posición donde estaba la base de la catapulta, se tenía que modificar también el brazo, la pelota, todo.

Para activar toda la animación se recibió la tecla F (Figura 3)

```
// tecla F
bool fAhora = mainWindow.getKeys()[GLFW_KEY_F];
if (fAhora && !fPresionadaAntes && estado == 0)
{
    ResetAnimacion(estado, tiempoEstado, rotacionBola, anguloBrazo, humoEscala, bola_t, bolaActiva, bolaPos, bolaEnHoyo);
    estado = 1;
}
fPresionadaAntes = fAhora;
```

Figura 3. Implementación tecla F para activación

Se agregaron transparencias, se instanciaron modelos y texturas con sus debidas transformaciones (Figuras 4 y 5)

```
// transparencias
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

// fuego
if (estado >= 1 && estado <= 3)
{
    float escalaFuego = 1.0f + 0.3f * sin(tiempo * 3.0f);
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    toffset = glm::vec2(0.0f, 0.0f);
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, -0.7f, 0.0f));
    model = glm::rotate(model, 90.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f * escalaFuego));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
    FireTexture.UseTexture();
    Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
    meshList[4] -> RenderMesh();
}

// humo
if (estado >= 3 && humoEscala > 0.0f)
{
    color = glm::vec3(1.0f, 1.0f, 1.0f);
    toffset = glm::vec2(0.0f, 0.0f);
    model = glm::mat4(1.0f);
    model = glm::translate(model, glm::vec3(1.0f, 3.0f, 0.0f));
    model = glm::rotate(model, -90.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
    model = glm::rotate(model, -90.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, -0.5f * (humoEscala - 1.0f)));
    model = glm::scale(model, glm::vec3(1.0f, 1.0f, humoEscala));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
    SmokeTexture.UseTexture();
    meshList[4] -> RenderMesh();
}
```

Figura 4. Transparencias y render de Fuego y Humo

```

// ===== RENDER PRÁCTICA 9 =====

// aeolipile
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
// base
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Aeolipile_base_M.RenderModel();
// bola giratoria
model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
model = glm::rotate(model, rotacionBola * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Aeolipile_M.RenderModel();

// catapulta
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
// base
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(5.0f, -2.0f, 0.0f));
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(3.5f, 3.5f, 3.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
CatapultaBase_M.RenderModel();
// brazo
glm::mat4 brazoPivot(1.0f);
brazoPivot = glm::translate(brazoPivot, glm::vec3(4.49f, -1.9f, 0.0f));
brazoPivot = glm::rotate(brazoPivot, 180.0f * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
brazoPivot = glm::rotate(brazoPivot, 30.0f * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
brazoPivot = glm::rotate(brazoPivot, -anguloBrazo * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(brazoPivot, glm::vec3(3.5f, 3.5f, 3.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CatapultaBrazo_M.RenderModel();

// canasta
{
    glm::vec4 cestoLocal(-3.7f, 4.9f, 0.2f, 1.0f);
    glm::vec3 cestoActual = glm::vec3(brazoPivot * cestoLocal);
    if (!bolaActiva || (bolaActiva && !bolaEnHoyo && bola_t <= deltaTime))
        bolaLanzada = cestoActual;
}
ActualizarBola(estado, bolaActiva, bola_t, bolaPos, bolaEnHoyo, bolaLanzada, deltaTime);

// pelota
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
if (estado == 6 && bolaActiva)
{
    model = glm::mat4(1.0f);
    model = glm::translate(model, bolaPos);
    model = glm::scale(model, glm::vec3(1.3f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bola_M.RenderModel();
}
else
{
    model = glm::translate(brazoPivot, glm::vec3(-3.7f, 4.9f, 0.2f));
    model = glm::scale(model, glm::vec3(1.3f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    Bola_M.RenderModel();
}

```

Figura 5. Render aeolipile, catapulta y pelota

La función principal, que maneja los estados fue la siguiente (Figura 6) se agregan comentarios que describen perfectamente el funcionamiento de cada estado.

```
void ActualizarEstados(int& estado, float& tiempoEstado,
    float& humoEscala, bool& bolaActiva,
    float& bola_t, float deltaTime)
{
    if (estado == 0) return;

    tiempoEstado += deltaTime;

    switch (estado)
    {
        case 1: // enciende fuego / espera / bola empieza a girar
            if (tiempoEstado >= 200.0f)
            {
                estado = 2;
                tiempoEstado = 0.0f;
            }
            break;

        case 2: // bola girando + fuego / espera / humo aparece
            if (tiempoEstado >= 200.0f)
            {
                estado = 3;
                tiempoEstado = 0.0f;
            }
            break;

        case 3: // humo crece + fuego / espera / fuego se apaga
            if (tiempoEstado >= 170.0f)
            {
                estado = 4;
                tiempoEstado = 0.0f;
            }
            break;

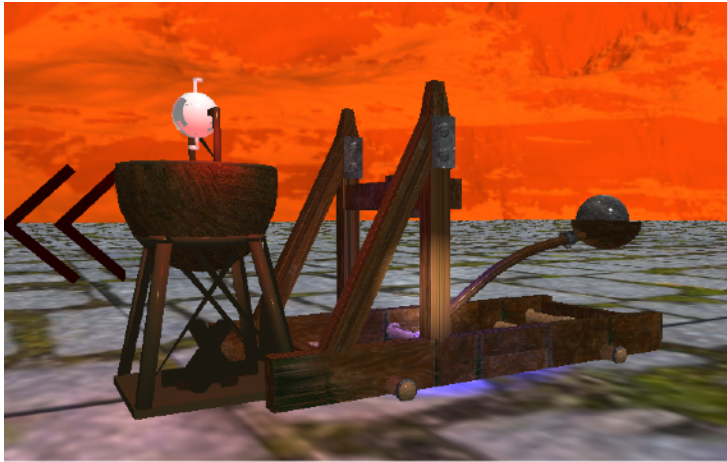
        case 4: // fuego apagado + humo crece hasta max
            break;

        case 5: // catapulta lanza en humo max
            estado = 6;
            tiempoEstado = 0.0f;
            bolaActiva = true;
            bola_t = 0.0f;
            break;

        case 6: // pelota en lanzamiento
            break;
    }
}
```

Figura 6. ActualizarEstados

Finalmente, este fue el resultado:



Inicio, sin activar nada



Se activa fuego (con tecla F)



Se activa movimiento de bola



Se activa humo

No pude captar el momento en que la catapulta lanza la pelota, en la última figura se ve que ya la lanzó, pero se adjunta [video que muestra el funcionamiento completo](#).

COMENTARIO:

Mi práctica tiene aspectos a mejorar, por ejemplo, al ejecutar existe un Delay entre el momento en el que el humo llega a su longitud máxima definida y el momento en el que se activa la catapulta, sin embargo, no lo pude resolver. Me resultó complicado ya que solo tuvimos una sesión de animación, considero que si se debería dedicar

más tiempo a la explicación, también fue complicado trabajar con la máquina de estados, sin embargo, considero que la práctica salió bien y con más tiempo podría salir mejor.